Talos

Implementation Plan for an Agent-Agnostic Coding Orchestration Platform

Prepared for Paul Bernard | Revision 2.1 | 2026-07-10

Purpose This document describes how to build a self-hosted, visual AI agent harness for managing coding work across many repositories, projects, portfolios, and agent providers. The platform does not build a new coding agent. It orchestrates existing agents such as Claude Code, OpenAI Codex, OpenCode, OpenHands, Gemini CLI, and similar tools through adapters, command-line harnesses, APIs, or protocols such as ACP/MCP where available.

Revision 2 resolves every open technology choice, fixes naming inconsistencies, and adds the contracts a less capable implementation model needs: full database DDL, REST request/response conventions, event payload schemas, the agent adapter interface in its implementation language, run/task state machines with ownership and timeouts, per-phase acceptance criteria, and an environment variable reference. Where this revision conflicts with earlier drafts, this revision wins.

Revision 2.1 (2026-07-10) expands the Section 16 Post-MVP outline into per-feature phase specifications (Phases 12–16). No MVP-scope contract (Sections 7–11) changed.

Table of Contents

1. Executive summary

Talos is a web-based control plane that manages projects, tasks, agent runs, workspaces, reviews, approvals, tests, pull requests, and deployment triggers. It does not attempt to build a new coding agent. The platform treats coding agents as replaceable execution engines behind a stable adapter interface.

> **Core decision** Use a single monorepo, but deploy multiple services. This gives one codebase for development and versioning, while keeping the web UI, API, orchestrator, and runner supervisor independently deployable in Dokploy.

1.1 Canonical naming

The platform name is Talos. Use Talos from the first commit across repository names, service names, package names, environment variables, Docker services, documentation titles, and default UI labels. Do not use previous working-title names anywhere. Before every commit, `grep -ri agentos .` must return zero results.

Codebase surface	Canonical Talos naming
Repository/root folder	`talos/`
Product/UI name	Talos
Java base package	`dev.talos`
Docker service names	`talos-api`, `talos-web`, `talos-orchestrator`, `talos-runner-supervisor`
Environment variable prefix	`TALOS_`
Database name	`talos`
Queue exchange	`talos.events` (routing keys like `task.run.requested`)
Project config file	`talos.yaml` (no legacy aliases in MVP)

1.2 MVP definition of done

The MVP is complete when every assertion below can be demonstrated on a fresh checkout with `docker compose -f infra/docker-compose.dev.yml up`:

1. A repository can be registered as a project through the UI, with a validated `talos.yaml` stack profile stored in `project_configs`.
2. A task can be created from the dashboard and moved across the Kanban board; illegal status transitions are rejected server-side.
3. Starting a run on a task publishes `task.run.requested` to RabbitMQ and creates an `agent_runs` row in status `QUEUED`.
4. The orchestrator consumes the request, the runner supervisor creates an isolated git worktree on branch `agent/task-<id>-<slug>`, and the selected adapter executes inside it.

5. Logs stream live from the runner to the browser through the internal API, Redis pub/sub, and Server-Sent Events.
6. The project's `talos.yaml` test command runs after the agent completes, and its result is recorded on the run.
7. The captured git diff and changed-file summary appear in the Review Center, with policy-risk flags where blocked patterns match.
8. Approving the run is required before any push, pull request, or deploy action; this gate is enforced server-side and covered by a test.
9. The end-to-end smoke test (`scripts/smoke.sh`) passes in CI: create project → create task → start run → CustomShellAdapter runs → diff captured → approval requested.

2. How this compares to existing tools

The architecture is similar in spirit to existing tools, but narrower and more operator-focused. The goal is not a general personal assistant and not just a coding UI. It is a governance and orchestration layer for multiple existing coding agents across multiple projects.

Tool / Pattern	What it already does	How Talos differs
OpenHands / Agent Canvas	Self-hostable coding-agent platform with browser UI, SDK, APIs, model-agnostic positioning, ACP support, Docker sandbox patterns.	Similar sandbox/runtime ideas, but with a stronger project dashboard, Kanban board, Dokploy deployment integration, and a provider-agnostic adapter layer. OpenHands can be one adapter/runtime, not the entire product.
Hermes / OpenClaw style assistants	Multi-channel assistant/gateway systems with skills, memories, MCP/ACP bridges, chat-based triggering.	Borrow the remote-trigger concept, but the center of gravity is software engineering: projects, branches, tests, PRs, approvals, deployments.
Vibe Kanban	Kanban issues, agent workspaces, branches, terminals, dev servers, diff review, app previews for coding agents.	Closest to the Kanban idea, but Talos is self-hosted around this stack, with a formal project registry, policy engine, Dokploy integration, and an explicit subscription/API execution strategy.
Nimbalyst	Visual local workspace for Codex, Claude Code, OpenCode: sessions, worktrees, visual editing, approvals, task management.	Useful UX inspiration, but Talos targets a remote web control plane hosted on a VPS, not only a local visual workspace.
BuilderMethods Agent OS	Coding standards and instructions for AI-powered development.	A documentation/standards idea, not a full runtime. Talos includes standards but also execution, orchestration, review, and deployment.
GitHub Agent HQ / Copilot agents	GitHub-native mission control for multiple agents and pull-request workflows.	Good reference point, but less self-hosted and less customizable around a VPS, Dokploy, Telegram/WhatsApp triggers, and local/subscription CLI modes.

Conclusion A separate front end, back end, database, queue, cache, orchestrator, and agent runners is consistent with how serious agent platforms are evolving. The product is built as a monorepo of services, not one giant runtime. Agent runners remain isolated because they execute untrusted or semi-trusted code and shell commands.

3. Product definition and non-goals

3.1 Product definition

Talos is a self-hosted, agent-agnostic development control plane for orchestrating coding agents across a portfolio of projects. It gives the user a central dashboard for project status, task planning, agent execution, review gates, and deployment triggers.

3.2 Primary users

- Solo technical founder managing several products.
- Freelancer managing multiple client repositories.
- Small dev agency using Claude Code, Codex, OpenCode, or other coding agents.
- Internal DevOps/IT team that wants AI coding automation with governance.

3.3 Non-goals

- Do not build a new LLM or coding model.
- Do not start by building a full IDE.
- Do not let agents work directly on production branches.
- Do not allow unattended production deployment in the MVP.
- Do not rely on subscription credentials as a SaaS business model unless the execution happens on the customer's own machine and complies with provider terms.
- Delete operations for projects and tasks are out of MVP scope; records are cancelled or archived via status, never hard-deleted.

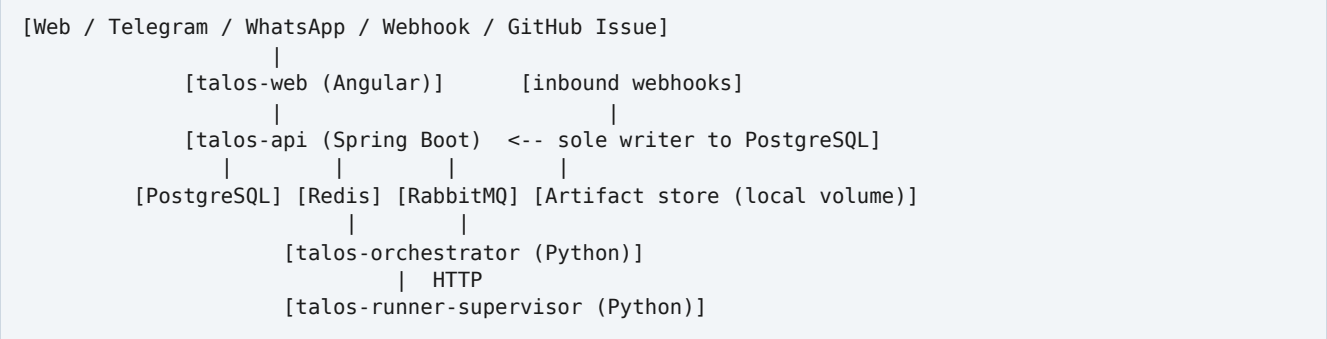
4. Target architecture

4.0 Pinned technology decisions

Every choice below is final for the MVP. Implementers must not substitute alternatives without an approved amendment to this document. Verify latest *patch* versions at implementation time; do not change major/minor pins.

Concern	Decision
Frontend	Angular 22, standalone components, signal-based state (no NgRx), Angular Material + CDK (drag-drop for Kanban), Node 22 LTS
Backend API	Spring Boot 4.1.x, Java 21 LTS, Maven (`./mvnw`), Spring Data JPA, Spring Security with JWT
Migrations	Flyway (bundled with Spring Boot), `V001__description.sql` naming, never edit an applied migration
Database	PostgreSQL 17 (pgvector extension deferred to post-MVP memory phase)
Queue	RabbitMQ 4.1 (topic exchange `talos.events`, quorum queues)
Cache / pub-sub	Redis 7 (locks, log fan-out)
Orchestrator	Python 3.12, `uv` for dependency management, `aio-pika` (RabbitMQ), `httpx` (API client), `pytest`
Runner supervisor	Python 3.12 (not Go), FastAPI + uvicorn, shares `packages/agent-adapter-spec` with the orchestrator
Contracts	OpenAPI 3.1 in `packages/contracts/openapi.yaml` (source of truth); JSON Schema per queue event in `packages/contracts/events/`
IDs	UUID v7, generated application-side
Timestamps	`TIMESTAMPTZ`, always UTC, ISO-8601 in APIs and events
Live updates	Server-Sent Events (SSE) fed from Redis pub/sub. WebSockets are not used in the MVP
Deployment	Docker images per service, deployed via Dokploy

4.1 System diagram



```
                          |
                [Agent adapter layer]
        /       |        |        |       \
CustomShell ClaudeCode OpenCode CodexCli OpenHands
                          |
         [Isolated git worktree / Docker workspace]
                          |
          [Build, Test, Review, PR, Deploy]
```

4.2 Service boundaries

Service	Technology	Responsibility
`talos-web`	Angular	Dashboard, Kanban, project registry, run viewer, diff/review UI, approval actions.
`talos-api`	Spring Boot	Auth, REST APIs, all persistence writes, task lifecycle, run state machine, approval workflow, integration registry, audit trail, SSE fan-out.
`talos-orchestrator`	Python	Consumes run requests, drives the run pipeline, selects adapters, calls runner supervisor, reports state/logs to the API. Holds no durable state.
`talos-runner-supervisor`	Python	Prepares git workspaces/worktrees, executes adapters as subprocesses (later in per-run containers), streams stdout/stderr, captures diffs.
`postgres`	PostgreSQL 17	System of record for users, projects, tasks, runs, approvals, audit.
`rabbitmq`	RabbitMQ 4.1	Durable async job and event queue.
`redis`	Redis 7	Run locks, log pub/sub channels, rate limits.
Artifact storage	Local volume first; MinIO/S3 later	Transcripts, patches, test reports, generated docs.

4.3 Inter-service communication contract

This is the spine of the system. There are exactly four communication paths:

1. **Browser → API.** REST over HTTPS with JWT (`Authorization: Bearer <token>`), plus one SSE endpoint per open run view.
2. **API → RabbitMQ → Orchestrator.** The API publishes events (Section 11); the orchestrator consumes them. The orchestrator never receives HTTP calls from the API.
3. **Orchestrator → API (`/internal/v1`).** *The API is the only writer to PostgreSQL.* The orchestrator and runner supervisor mutate state exclusively through the internal REST namespace, authenticated with the shared service token `TALOS_INTERNAL_API_TOKEN`. Neither Python service opens a database connection.
4. **Orchestrator → Runner supervisor.** Plain HTTP on the internal Docker network: `POST /workspaces/prepare`, `POST /runs/{id}/execute` (chunked streaming response carrying adapter events), `POST /runs/{id}/stop`, `GET /health`.

Log flow (authoritative): runner supervisor streams line-buffered stdout/stderr to the orchestrator over the chunked `execute` response → orchestrator batches lines (flush at 50 lines or 2 seconds) → `POST /internal/v1/runs/{id}/logs` → API persists to `agent_run_logs` and publishes each batch to Redis channel `talos:run:{run_id}:logs` → `GET /api/v1/runs/{id}/events/stream` (SSE) relays to the browser.

5. Monorepo and deployment model

One codebase, multiple runtimes. The agent runner must be isolated from the API and web UI because it runs shell commands, clones repositories, and executes potentially unsafe code.

```
talos/
  apps/
    web/                    # Angular 22 dashboard
    api/                    # Spring Boot 4.1 API (dev.talos)
    orchestrator/           # Python orchestration service
    runner-supervisor/      # Python workspace/adapter executor
    telegram-adapter/       # Post-MVP (Phase 12)
    whatsapp-adapter/       # Post-MVP (Phase 12)
  workers/
    base-agent-runner/      # Base Docker image for per-run containers (Phase 11)
    java-runner/            # Stack image: JDK 21 + Maven
    node-runner/            # Stack image: Node 22
    python-runner/          # Stack image: Python 3.12 + uv
  packages/
    contracts/              # openapi.yaml + events/*.json (source of truth)
    project-config-schema/  # talos.schema.json for talos.yaml
    agent-adapter-spec/     # Python AgentAdapter ABC, dataclasses, contract tests
  infra/
    docker-compose.dev.yml
    dokploy/                # Deployment runbook and service definitions
  scripts/
    smoke.sh                # End-to-end smoke test
  docs/
    architecture.md
    security-model.md
    agent-run-lifecycle.md
    provider-auth.md
    deployment.md
    phase-reports/          # phase-N-report.md written at each phase gate
```

Odoo and Flutter runner images are deferred until a real project needs them; do not scaffold empty directories for them.

6. Core services

6.1 Angular web dashboard

The dashboard is the main product surface. Modules and routes are specified in Section 15. State management rules: one signal-based store service per domain (`ProjectStore`, `TaskStore`, `RunStore`), components read signals and call store methods, the generated OpenAPI client is the only HTTP access path.

Module	MVP behavior
Command Center	All projects, running agent jobs, blocked approvals, recent failures, suggested next actions.
Project Registry	Create/update project records, repo URLs, stack type, build/test commands, deploy targets, risk rules.

Module	MVP behavior
Kanban Board	Columns: Backlog, Ready, Running, Review, Blocked, Done. Cards map to tasks and can start agent runs. Cancelled tasks appear only in filtered lists, never as a column.
Agent Runs	Live log stream, status timeline, current step, model/agent used, artifacts.
Review Center	Diff viewer, changed files, test results, risk flags, approve/reject/request-changes.
Integrations	Configure GitHub, Dokploy, provider credentials, local CLI modes.
Memory/Docs	Post-MVP (Phase 13): project conventions, run history, decision log.

6.2 Spring Boot control API

The API is the authoritative control plane. It owns persistent state, authentication, permissions, task lifecycle, the run state machine, approval workflow, and external integration metadata. **No other service writes to PostgreSQL.**

```
dev.talos.common          # error envelope, pagination, UUID v7 generator
dev.talos.auth            # login, JWT filter, seeded admin
dev.talos.projects        # projects + project_configs + talos.yaml parsing
dev.talos.tasks           # tasks CRUD, Kanban transitions
dev.talos.runs            # run state machine, internal ingest, SSE emitter, reaper
dev.talos.approvals       # approval workflow, policy scan results
dev.talos.policy          # policy.yaml loading, diff/command scanning
dev.talos.integrations    # GitHub, Dokploy clients behind interfaces
dev.talos.secrets         # AES-256-GCM encrypted secret store
dev.talos.notifications   # approval reminders (log-only in MVP)
dev.talos.audit           # audit_events writer
dev.talos.webhooks        # inbound webhook endpoints (GitHub in MVP)
dev.talos.events          # RabbitMQ publisher, event envelope
```

Package convention inside each module: `XController` (REST), `XService` (business logic), `XRepository` (Spring Data), `dto/` (request/response records — entities never leave the service layer).

6.3 Python orchestrator

Stateless coordinator. All durable state lives in PostgreSQL behind the API. Layout:

```
apps/orchestrator/
  pyproject.toml            # uv-managed
  src/talos_orchestrator/
    main.py                 # aio-pika consumer bootstrap
    pipeline.py             # the run pipeline (one class, one run at a time per message)
    api_client.py           # httpx client for /internal/v1
    runner_client.py        # httpx client for runner supervisor
    adapters.py             # adapter selection from registry
    locks.py                # Redis lock helpers
  tests/
```

Responsibilities, in pipeline order: consume `task.run.requested` (prefetch 1, manual ack) → acquire Redis lock `talos:lock:run:{project_id}:{base_branch}` (TTL = run timeout; reject with run FAILED reason `CONCURRENT_RUN` if held) → load project/config/task via internal API → transition run through its states (Section 8.2), calling the runner supervisor for workspace prep and

adapter execution → forward log batches → trigger the `talos.yaml` test command via the runner → request the diff capture → post changes and summary → move run to `WAITING_APPROVAL` → ack the message. On any exception: set run `FAILED` with `error_message`, release the lock, ack (the message is not retried; retry is a human action from the UI).

Crash recovery: if the orchestrator dies mid-run, the unacked message redelivers; the pipeline treats a run not in `QUEUED` as poisoned, marks it `FAILED` with reason `ORPHANED_BY_RESTART`, and acks. The API-side reaper (Section 8.2) independently fails any run whose `timeout_at` has passed.

6.4 Runner supervisor

FastAPI service owning the filesystem. Endpoints: `POST /workspaces/prepare` (clone or fetch, create worktree + branch, returns `workspace_path`), `POST /runs/{id}/execute` (body: adapter key, prompt, env allowlist, timeout; response: chunked stream of adapter events), `POST /runs/{id}/tests` (runs configured command in workspace, streams output, returns exit code), `POST /runs/{id}/diff` (returns changed-file summary + unified diff, also writes `diff.patch` artifact), `POST /runs/{id}/stop` (SIGTERM process group, SIGKILL after 10 s), `POST /workspaces/cleanup` (retention enforcement), `GET /health`. MVP runs adapters as local subprocesses under resource limits; Phase 11 moves execution into per-run Docker containers using the `workers/` images without changing this HTTP contract.

7. Agent adapter layer

The most important abstraction. Every agent provider looks identical to the orchestration layer, whether implemented as an API call, CLI process, ACP server, or MCP bridge. Adapters are implemented **in Python** (they execute inside the runner supervisor) and live in `packages/agent-adapter-spec`.

7.1 The AgentAdapter contract

```
from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from enum import Enum
from typing import AsyncIterator

class AgentEventType(str, Enum):
    LOG = "log"            # raw stdout/stderr line
    TOOL_USE = "tool_use"  # structured action (command, file edit) when parseable
    STATUS = "status"      # adapter-level status change
    ERROR = "error"

@dataclass
class ProviderCapabilities:
    supports_streaming: bool
    supports_subscription_auth: bool
    supports_api_key_auth: bool
    supports_headless_mode: bool
    supports_diff_output: bool
    supports_approval_hooks: bool
    default_timeout_seconds: int = 1800

@dataclass
class AgentSessionRequest:
    run_id: str
    workspace_path: str        # absolute path to the worktree
    prompt: str                # fully assembled prompt (see 7.3)
    env: dict[str, str]        # ONLY approved injected variables
    auth_mode: str             # "api_key" | "subscription_local"
    provider_home: str         # isolated HOME dir holding provider credentials
    timeout_seconds: int
    container: ContainerConfig | None = None  # Phase 11: when set, run in a per-run Docker container
(Section 8) instead of a local subprocess

@dataclass
class AgentEvent:
    type: AgentEventType
    message: str
    timestamp: str             # ISO-8601 UTC
    metadata: dict = field(default_factory=dict)

@dataclass
class AgentResult:
    exit_code: int
    success: bool
    summary: str | None
    raw_output_path: str       # artifact file with the full transcript

class AgentAdapter(ABC):
    key: str  # "custom-shell" | "claude-code" | "opencode" | "codex-cli" | ...
```

```
    @abstractmethod
    def capabilities(self) -> ProviderCapabilities: ...

    @abstractmethod
    async def start(self, request: AgentSessionRequest) -> None: ...

    @abstractmethod
    def events(self) -> AsyncIterator[AgentEvent]: ...

    @abstractmethod
    async def stop(self) -> None:
        """SIGTERM the process group; SIGKILL survivors after 10 seconds."""

    @abstractmethod
    async def result(self) -> AgentResult: ...
```

7.2 Adapter contract tests

Every adapter must pass the shared suite in `packages/agent-adapter-spec/tests/contract/` before it may be registered:

1. `start()` followed by `events()` yields at least one event.
2. The adapter terminates within `timeout_seconds` and reports `success=False` on timeout.
3. `stop()` leaves no child processes (verified via process group inspection).
4. `result().exit_code` matches the underlying process exit code.
5. The adapter writes nothing outside `workspace_path` and `provider_home`.
6. No value from `request.env` ever appears in emitted events (masking check).

7.3 Prompt assembly

Prompts are assembled by the orchestrator, not by adapters, in this fixed order: (1) a constraints preamble ("You are working in an isolated branch of . Do not modify files matching: . Do not run destructive commands. Stop when the task is complete."), (2) project context — the contents of each file listed in `talos.yaml context.docs`, truncated to 8,000 characters each, (3) the task title and description verbatim, (4) the expected deliverable ("Make the necessary code changes. Do not commit; Talos handles commits."). The assembled prompt is stored on `agent_runs.prompt` for auditability.

7.4 Adapter implementations and order

Implementation order is fixed. CustomShellAdapter proves the pipeline; ClaudeCodeAdapter is the first real agent (it is the operator's daily driver, so runs can be verified immediately, and its headless mode and hook system are well documented).

#	Adapter	Approach	Notes
1	`CustomShellAdapter` (Phase 6)	Executes a configured deterministic script in the workspace	No AI. Makes the entire lifecycle testable in CI. Also reused for build/test/deploy steps.

#	Adapter	Approach	Notes
2	`ClaudeCodeAdapter` (Phase 7)	Headless CLI: `claude -p "<prompt>" --output-format stream-json --verbose --max-turns 50 --permission-mode acceptEdits` with `cwd=workspace_path`, `HOME=provider_home`, `CLAUDE_CONFIG_DIR=provider_home/.claude`	Parses stream-json events into `TOOL_USE`/`LOG` events. Subscription auth: run `claude login` once inside the provider home. API-key auth: set `ANTHROPIC_API_KEY` in `env`. Use Claude Code permission settings/hooks as a native policy enforcement point. Do not build a SaaS around customer subscription login unless provider-approved. Verify exact flags against current Claude Code docs at implementation time.
3	`OpenCodeAdapter` (Phase 12)	CLI/server wrapper	Open source and provider-flexible; keeps the agent-agnostic claim honest.
4	`CodexCliAdapter` (Phase 12)	`codex exec` non-interactive mode; API-key mode for automation	ChatGPT sign-in acceptable for personal use only.
5	`OpenHandsAdapter` (Phase 12+)	HTTP/API/ACP integration	Optional sandboxed execution backend.
6	`GeminiCliAdapter` (backlog)	CLI/ACP where available	Lower priority.

Until an adapter's phase arrives, it exists only as a stub class raising `NotImplementedError` with a `TODO` comment. Agent CLIs change quickly: each real adapter's first implementation task is to verify current invocation flags against the provider's documentation, and each adapter documents its verified CLI version in its module docstring.

7.5 Provider credential isolation

Each provider gets an isolated home directory under `/var/talos/provider-homes/<agent-key>/` (a Docker volume). Credentials (`claude login` session, API keys) live only there — never inside any workspace, never in the repository, never in logs. The runner supervisor sets `HOME` (and provider-specific config-dir variables) to the provider home when spawning the adapter process.

8. Workspace and execution model

Every agent run executes inside an isolated workspace. The MVP uses git worktrees plus subprocess resource limits; Phase 11 adds per-run Docker containers. Later, stronger isolation can use rootless Docker, Firecracker, or gVisor.

8.1 Run lifecycle

1. Receive `task.run.requested`.
2. Validate project, task, config, and Redis concurrency lock.
3. Prepare workspace: clone or fetch the repo, create worktree with branch `agent/task-<task-id>-<slug>`.
4. Inject only approved secrets as environment variables (Section 12).
5. Start the selected agent adapter with the assembled prompt.
6. Stream logs and events; persist via internal API.
7. On agent completion, run the project's `talos.yaml` test command.
8. Capture git status, changed-file summary, and unified diff; store `diff.patch` artifact.
9. Run the policy scan over the diff and captured commands; set risk flags.
10. Create the approval request; run enters `WAITING_APPROVAL`.
11. On approval: Talos commits, pushes, and opens the PR (Section 8.4). On rejection: task returns to Ready with reviewer notes.
12. Cleanup job removes terminal run workspaces older than the retention period.

8.2 Run state machine

```
CREATED → QUEUED → PREPARING_WORKSPACE → RUNNING_AGENT → RUNNING_TESTS
        → REVIEWING → WAITING_APPROVAL → APPROVED → COMPLETED

Failure edge:  any non-terminal state → FAILED   (error_message required)
Cancel edge:   QUEUED … WAITING_APPROVAL → CANCELLED (user action)
Reject edge:   WAITING_APPROVAL → REJECTED        (task returns to READY)
```

Transition	Owner	Timeout (moves run to FAILED)
CREATED → QUEUED	API, on `start-run`	—
QUEUED → PREPARING_WORKSPACE	Orchestrator	10 min in queue
PREPARING_WORKSPACE → RUNNING_AGENT	Orchestrator	5 min
RUNNING_AGENT → RUNNING_TESTS	Orchestrator	adapter timeout (default 30 min)
RUNNING_TESTS → REVIEWING	Orchestrator	20 min
REVIEWING → WAITING_APPROVAL	Orchestrator	5 min

Transition	Owner	Timeout (moves run to FAILED)
WAITING_APPROVAL → APPROVED / REJECTED	API, human decision	none (reminder event after 24 h)
APPROVED → COMPLETED	Orchestrator (commit/push/PR)	10 min
any → FAILED / CANCELLED	Orchestrator, reaper, or API	—

The API validates every transition against this table and rejects illegal ones with HTTP 422 plus an audit event. A scheduled **reaper** in `dev.talos.runs` (every 60 s) fails runs whose `timeout_at` has passed and releases their Redis locks.

Task ↔ run status mapping (applied by the API on each run transition): run `QUEUED`/`RUNNING_*` ⇒ task `RUNNING`; run `WAITING_APPROVAL` ⇒ task `REVIEW`; run `COMPLETED` ⇒ task `DONE`; run `REJECTED` ⇒ task `READY`; run `FAILED` ⇒ task `BLOCKED`; run `CANCELLED` ⇒ task returns to its status before the run.

8.3 Workspace directory structure

```
/var/talos/workspaces/
  <project-slug>/
    repo/                       # primary clone; fetched, never built in
    runs/
      <run-id>/
        worktree/               # git worktree, branch agent/task-<task-id>-<slug>
        artifacts/              # transcript.txt, diff.patch, test-report.*
        run.json                # run metadata snapshot
/var/talos/provider-homes/
  <agent-key>/                  # isolated HOME per provider (credentials only)
```

Cleanup: delete `runs/<run-id>` when the run is terminal AND older than `TALOS_MAX_WORKSPACE_AGE_DAYS` (default 7) — but never while the run's branch has an open pull request.

8.4 Git workflow

- **Credentials:** per-project deploy key or fine-grained PAT, stored as a secret ref, injected into the runner only via `GIT_SSH_COMMAND`/credential helper at clone/push time. Never embedded in remote URLs, never written into the workspace, never logged.
- **Who commits:** Talos commits, not the agent. The prompt instructs agents not to commit. After approval, the runner stages all changes and commits as author `Talos Agent <talos@local>` with message `talos: <task title> (task <task-id>, run <run-id>)`. If an agent committed anyway, those commits are preserved and Talos adds a final commit for uncommitted changes.
- **Push and PR:** push happens only after approval, immediately followed by PR creation via the GitHub REST API (fine-grained token with `contents:write`, `pull_requests:write` on the target repo). PR body template: task link, run link, diffstat, test results, "Generated by Talos" footer. PR URL and number stored in `pull_requests`.

- **Conflicts:** the worktree branches from the latest `default_branch` at prepare time. If push is rejected (non-fast-forward) or GitHub reports conflicts, the run is flagged `NEEDS_REBASE` in review notes; Talos never auto-merges or force-pushes. `force_push` to any branch and any push to `default_branch` are hard-blocked in the runner's git wrapper.

8.5 Execution risks and required guardrails

Risk	Required guardrail
Agent deletes important files	All runs happen on task branches/worktrees; deletions surface in diff review.
Agent reads secrets	`.env` files are never copied into worktrees (copy-filter honors `talos.yaml` `ignore_paths` plus a built-in `.env*/*.pem/id_*` filter); scoped secret injection only.
Agent runs destructive command	Container capability limits (Phase 11), agent-native permission hooks where supported, post-run command-log scan (Section 12.1).
Agent affects host	Non-root execution, no Docker socket in any runner container, resource limits (`ulimit`/cgroup: CPU, memory, pids, disk).
Run consumes VPS resources	`TALOS_MAX_ACTIVE_RUNS=1` initially, per-run timeout, workspace retention job.
Provider credentials leak	Isolated provider homes, secret masking in all log paths, contract test 7.2(6).

9. Data model

9.1 Conventions

UUID v7 primary keys generated application-side. `TIMESTAMPTZ` everywhere, UTC. `created_at NOT NULL DEFAULT now()`; `updated_at` maintained by the API on every mutation of mutable tables. Enums are `VARCHAR` columns with `CHECK` constraints matching Section 9.3 exactly. Foreign keys are declared and indexed. Flyway migrations: `V001__users_and_audit.sql`, `V002__core_schema.sql`, ...

9.2 MVP schema (DDL)

```
CREATE TABLE users (
  id UUID PRIMARY KEY,
  email VARCHAR(320) NOT NULL UNIQUE,
  name VARCHAR(200) NOT NULL,
  password_hash VARCHAR(100) NOT NULL,
  role VARCHAR(20) NOT NULL CHECK (role IN ('OWNER','MAINTAINER','REVIEWER','VIEWER')),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE audit_events (
  id UUID PRIMARY KEY,
  actor_user_id UUID REFERENCES users(id),      -- NULL for system actors
  event_type VARCHAR(100) NOT NULL,
  entity_type VARCHAR(50) NOT NULL,
  entity_id UUID,
  details_json JSONB NOT NULL DEFAULT '{}',
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_audit_entity ON audit_events(entity_type, entity_id);

CREATE TABLE projects (
  id UUID PRIMARY KEY,
  name VARCHAR(200) NOT NULL,
  slug VARCHAR(100) NOT NULL UNIQUE,
  repo_url VARCHAR(500) NOT NULL,
  default_branch VARCHAR(200) NOT NULL DEFAULT 'main',
  stack_type VARCHAR(50) NOT NULL,              -- 'spring-boot','angular','python',...
  status VARCHAR(20) NOT NULL DEFAULT 'ACTIVE' CHECK (status IN ('ACTIVE','ARCHIVED')),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE project_configs (
  id UUID PRIMARY KEY,
  project_id UUID NOT NULL REFERENCES projects(id),
  config_yaml TEXT NOT NULL,
  parsed_json JSONB NOT NULL,
  version INT NOT NULL,
  is_active BOOLEAN NOT NULL DEFAULT true,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (project_id, version)
);
-- application invariant: at most one is_active=true row per project

CREATE TABLE tasks (
  id UUID PRIMARY KEY,
```

```
  project_id UUID NOT NULL REFERENCES projects(id),
  title VARCHAR(300) NOT NULL,
  description TEXT,
  source VARCHAR(30) NOT NULL DEFAULT 'DASHBOARD',  -- DASHBOARD|WEBHOOK|TELEGRAM|...
  status VARCHAR(20) NOT NULL DEFAULT 'BACKLOG'
    CHECK (status IN ('BACKLOG','READY','RUNNING','REVIEW','BLOCKED','DONE','CANCELLED')),
  priority VARCHAR(10) NOT NULL DEFAULT 'MEDIUM' CHECK (priority IN ('LOW','MEDIUM','HIGH')),
  risk_level VARCHAR(10) NOT NULL DEFAULT 'NORMAL' CHECK (risk_level IN ('NORMAL','HIGH')),
  board_position INT NOT NULL DEFAULT 0,             -- ordering within a Kanban column
  requested_by UUID REFERENCES users(id),
  assigned_agent_key VARCHAR(50),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_tasks_project_status ON tasks(project_id, status);

CREATE TABLE agent_runs (
  id UUID PRIMARY KEY,
  task_id UUID NOT NULL REFERENCES tasks(id),
  project_id UUID NOT NULL REFERENCES projects(id),
  status VARCHAR(30) NOT NULL DEFAULT 'CREATED'
    CHECK (status IN ('CREATED','QUEUED','PREPARING_WORKSPACE','RUNNING_AGENT',
      'RUNNING_TESTS','REVIEWING','WAITING_APPROVAL','APPROVED','REJECTED',
      'COMPLETED','FAILED','CANCELLED')),
  agent_key VARCHAR(50) NOT NULL,
  provider_auth_mode VARCHAR(30) NOT NULL DEFAULT 'api_key',
  prompt TEXT,
  branch_name VARCHAR(300),
  workspace_path VARCHAR(500),
  summary TEXT,
  diff_patch TEXT,                                   -- Phase 8: unified diff text (Section 8.1 step
8), served by GET /api/v1/runs/{id}/diff
  test_status VARCHAR(20) NOT NULL DEFAULT 'NOT_RUN'
    CHECK (test_status IN ('NOT_RUN','PASSED','FAILED','ERROR')),
  review_status VARCHAR(20) NOT NULL DEFAULT 'CLEAN'
    CHECK (review_status IN ('CLEAN','RISK_FLAGGED')),
  error_message TEXT,
  exit_code INT,
  timeout_at TIMESTAMPTZ,
  started_at TIMESTAMPTZ,
  completed_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_runs_task ON agent_runs(task_id);
CREATE INDEX idx_runs_status ON agent_runs(status);

CREATE TABLE agent_run_steps (
  id UUID PRIMARY KEY,
  run_id UUID NOT NULL REFERENCES agent_runs(id),
  step_type VARCHAR(20) NOT NULL
    CHECK (step_type IN ('WORKSPACE','AGENT','TESTS','REVIEW','PUSH','PR','DEPLOY')),
  status VARCHAR(20) NOT NULL CHECK (status IN ('RUNNING','COMPLETED','FAILED','SKIPPED')),
  summary TEXT,
  started_at TIMESTAMPTZ,
  completed_at TIMESTAMPTZ
);

CREATE TABLE agent_run_logs (
  id UUID PRIMARY KEY,
  run_id UUID NOT NULL REFERENCES agent_runs(id),
  step_id UUID REFERENCES agent_run_steps(id),
  sequence BIGINT NOT NULL,
  stream VARCHAR(10) NOT NULL CHECK (stream IN ('STDOUT','STDERR','SYSTEM')),
```

```
  message TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (run_id, sequence)
);

CREATE TABLE approvals (
  id UUID PRIMARY KEY,
  task_id UUID NOT NULL REFERENCES tasks(id),
  run_id UUID NOT NULL REFERENCES agent_runs(id),
  approval_type VARCHAR(30) NOT NULL,           -- 'RUN_RESULT','DEPLOY',...
  requested_action VARCHAR(200) NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'PENDING'
    CHECK (status IN ('PENDING','APPROVED','REJECTED','CHANGES_REQUESTED','EXPIRED')),
  requested_by UUID REFERENCES users(id),
  approved_by UUID REFERENCES users(id),
  approved_at TIMESTAMPTZ,
  notes TEXT,
  expires_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  environment VARCHAR(50)                           -- Phase 10: which environment a DEPLOY-type
approval is for
);

CREATE TABLE git_changes (
  id UUID PRIMARY KEY,
  run_id UUID NOT NULL REFERENCES agent_runs(id),
  file_path VARCHAR(1000) NOT NULL,
  change_type VARCHAR(10) NOT NULL CHECK (change_type IN ('ADDED','MODIFIED','DELETED','RENAMED')),
  additions INT NOT NULL DEFAULT 0,
  deletions INT NOT NULL DEFAULT 0,
  risk_flagged BOOLEAN NOT NULL DEFAULT false,
  matched_pattern VARCHAR(200)                      -- Phase 8: the policy.yaml/talos.yaml pattern
that flagged this file, shown in the Review Center
);

CREATE TABLE pull_requests (
  id UUID PRIMARY KEY,
  run_id UUID NOT NULL REFERENCES agent_runs(id),
  provider VARCHAR(20) NOT NULL DEFAULT 'github',
  pr_number INT,
  url VARCHAR(500),
  status VARCHAR(20) NOT NULL DEFAULT 'OPEN' CHECK (status IN ('OPEN','MERGED','CLOSED')),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE integrations (
  id UUID PRIMARY KEY,
  type VARCHAR(30) NOT NULL,                    -- 'github','dokploy','anthropic',...
  name VARCHAR(100) NOT NULL,
  config_json JSONB NOT NULL DEFAULT '{}',      -- non-secret settings only
  enabled BOOLEAN NOT NULL DEFAULT true,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE secret_values (
  id UUID PRIMARY KEY,
  encrypted_value BYTEA NOT NULL,               -- AES-256-GCM via TALOS_SECRETS_KEY
  nonce BYTEA NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
-- read/written ONLY by dev.talos.secrets; never exposed via REST

CREATE TABLE integration_credentials (
```

```
  id UUID PRIMARY KEY,
  integration_id UUID NOT NULL REFERENCES integrations(id),
  secret_ref UUID NOT NULL REFERENCES secret_values(id),
  auth_mode VARCHAR(30) NOT NULL,               -- 'api_key','pat','deploy_key',...
  owner_user_id UUID REFERENCES users(id),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Phase 10 (Section 9.4 deferred this to here): one row per project+environment deploy target,
-- synced from talos.yaml's deploy: block, tracking the most recent Dokploy deployment's status.
CREATE TABLE project_environments (
  id UUID PRIMARY KEY,
  project_id UUID NOT NULL REFERENCES projects(id),
  environment VARCHAR(50) NOT NULL,
  provider VARCHAR(30) NOT NULL DEFAULT 'dokploy',
  app_id VARCHAR(200) NOT NULL,
  approval_required BOOLEAN NOT NULL DEFAULT true,
  last_deploy_status VARCHAR(20) CHECK (last_deploy_status IN ('RUNNING','SUCCEEDED','FAILED')),
  last_deployed_at TIMESTAMPTZ,
  last_run_id UUID REFERENCES agent_runs(id),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (project_id, environment)
);
```

9.3 Enum reference (single source of truth)

```
TaskStatus:     BACKLOG, READY, RUNNING, REVIEW, BLOCKED, DONE, CANCELLED
RunStatus:      CREATED, QUEUED, PREPARING_WORKSPACE, RUNNING_AGENT, RUNNING_TESTS,
                REVIEWING, WAITING_APPROVAL, APPROVED, REJECTED, COMPLETED, FAILED, CANCELLED
ApprovalStatus: PENDING, APPROVED, REJECTED, CHANGES_REQUESTED, EXPIRED
TestStatus:     NOT_RUN, PASSED, FAILED, ERROR
ReviewStatus:   CLEAN, RISK_FLAGGED
StepType:       WORKSPACE, AGENT, TESTS, REVIEW, PUSH, PR, DEPLOY
LogStream:      STDOUT, STDERR, SYSTEM
Role:           OWNER, MAINTAINER, REVIEWER, VIEWER
DeployStatus:   RUNNING, SUCCEEDED, FAILED (Phase 10; null on project_environments = never deployed)
```

Note: `APPROVED` and `REJECTED` are real run states (absent from earlier drafts); without them, "approved but push failed" is unrepresentable.

9.4 Deferred tables (do NOT create in MVP migrations)

`project_environments` was deferred here through Phase 9; it's created in Phase 10's `V004__deploy.sql` (see Section 9.2). `memory_documents` and `memory_chunks` (+ pgvector extension, Phase 13) remain deferred. Listing them here prevents an implementer from guessing them into `V002`.

10. API surface

10.1 Conventions

- Base paths: `/api/v1` (public, JWT via `Authorization: Bearer`), `/internal/v1` (service token via `X-Talos-Internal-Token`), `/api/v1/webhooks/*` (HMAC signature verification, no JWT).
- Error envelope, all non-2xx responses: `{"error": {"code": "TASK_INVALID_TRANSITION", "message": "...", "details": {}}}`. Codes are SCREAMING_SNAKE strings enumerated in the OpenAPI spec.
- List endpoints: `?page=0&size=20` plus documented filters; response `{"content": [...], "page": 0, "size": 20, "totalElements": n}`.
- `packages/contracts/openapi.yaml` is the source of truth: the Angular client is generated from it (`openapi-generator`, `typescript-angular`), and a CI check diffs the running API's springdoc output against it.

10.2 Public endpoints

Method + path	Request → Response
`POST /api/v1/auth/login`	`{email, password}` → `{token, expiresAt}`
`GET /api/v1/projects`	filters `status` → page of ProjectSummary
`POST /api/v1/projects`	`{name, repoUrl, defaultBranch, stackType}` → Project (201)
`GET /api/v1/projects/{id}`	→ Project + activeConfig + last 5 runs
`PUT /api/v1/projects/{id}`	Project fields → Project
`POST /api/v1/projects/{id}/sync-config`	`{configYaml}` → ProjectConfig (validates against talos.schema.json; 422 with field errors on failure)
`GET /api/v1/projects/{id}/runs`	→ page of RunSummary
`GET /api/v1/tasks`	filters `projectId,status` → page of Task
`POST /api/v1/tasks`	`{projectId, title, description, priority, riskLevel}` → Task (201)
`GET /api/v1/tasks/{id}`	→ Task + runs
`PATCH /api/v1/tasks/{id}`	partial Task → Task
`POST /api/v1/tasks/{id}/move`	`{status, boardPosition}` → Task (422 on illegal transition)
`POST /api/v1/tasks/{id}/start-run`	`{agentKey?, authMode?}` → Run (201; defaults from talos.yaml; 409 if active run exists)
`GET /api/v1/runs`	filters `projectId,status` → page of RunSummary
`GET /api/v1/runs/{id}`	→ Run + steps
`GET /api/v1/runs/{id}/logs`	`?afterSequence=` → log page

Method + path	Request → Response
`GET /api/v1/runs/{id}/events/stream`	SSE (Section 10.3)
`GET /api/v1/runs/{id}/diff`	→ `{files: [GitChange], diff: "unified text"}`
`GET /api/v1/runs/{id}/pull-request`	Phase 9 → PullRequest (404 until PullRequestService has opened one)
`POST /api/v1/runs/{id}/cancel`	→ Run (CANCELLED)
`POST /api/v1/runs/{id}/rerun-tests`	→ 202
`GET /api/v1/approvals`	filters `status` → page of Approval
`GET /api/v1/approvals/{id}`	→ Approval + run + diff summary
`POST /api/v1/approvals/{id}/approve`	`{notes?}` → Approval
`POST /api/v1/approvals/{id}/reject`	`{notes}` → Approval
`POST /api/v1/approvals/{id}/request-changes`	`{notes}` → Approval
`GET /api/v1/integrations`	→ list (secrets never returned)
`POST /api/v1/integrations`	`{type, name, configJson, secret?}` → Integration
`POST /api/v1/integrations/{id}/test`	→ `{ok, message}`
`POST /api/v1/webhooks/github`	GitHub webhook (HMAC-verified) → 204

There are no DELETE endpoints in the MVP (see non-goals).

10.3 SSE stream

`GET /api/v1/runs/{id}/events/stream` emits named events: `log` (`{sequence, stream, message, timestamp}`), `status` (`{from, to}`), `step` (`{stepType, status}`). Heartbeat comment every 15 s. The client reconnects with `Last-Event-ID` = last log sequence; the API backfills from `agent_run_logs` before resuming live relay from Redis channel `talos:run:{id}:logs`.

10.4 Internal endpoints (service token required)

```
POST /internal/v1/runs/{id}/status      {status, errorMessage?}      # validates state machine
POST /internal/v1/runs/{id}/steps       {stepType, status, summary?}
POST /internal/v1/runs/{id}/logs        {entries: [{stream, sequence, message, timestamp}]}
POST /internal/v1/runs/{id}/changes     {files: [...], diffArtifactRef}
POST /internal/v1/runs/{id}/artifacts   multipart {name, file}
GET  /internal/v1/runs/{id}/context     → run + task + project + active parsed config
GET  /internal/v1/runs/{id}/git-token   → {token, authMode, repoUrl, defaultBranch}   # Phase 9; 409
unless run APPROVED
POST /internal/v1/runs/{id}/pull-request {branchName, commitSha} → PullRequest         # Phase 9;
opens the PR, completes the run
```

10.5 Runner supervisor endpoints (called only by orchestrator)

Listed in Section 6.4; they are part of `packages/contracts` as a second OpenAPI file `runner-api.yaml`.

11. Event and queue design

Exchange: **`talos.events`** (topic, durable). Every message uses this envelope:

```
{
  "event_id": "uuid-v7",
  "event_type": "task.run.requested",
  "occurred_at": "2026-07-09T12:00:00Z",
  "version": 1,
  "payload": { }
}
```

JSON Schemas for each payload live in `packages/contracts/events/<event_type>.json` and are validated by producer (API) and consumer (orchestrator) tests.

Routing key	Producer → Consumer (queue)	Payload
`task.run.requested`	API → orchestrator (`talos.orchestrator.run-requests`)	`run_id, task_id, project_id, agent_key, auth_mode`
`approval.decided`	API → orchestrator (`talos.orchestrator.approvals`)	`approval_id, run_id, status, decided_by`
`run.status.changed`	API → (observability/notifiers)	`run_id, from, to`
`approval.requested`	API → notifiers	`approval_id, run_id, task_title, review_status`
`pr.created`	API → notifiers	`run_id, pr_url, pr_number`
`deploy.requested` / `deploy.completed` / `deploy.failed`	API/orchestrator (Phase 10)	`run_id, project_id, environment, dokploy_app_id[, error]`

Rules: quorum queues, manual ack, prefetch 1. A message that fails processing is retried up to 3 redeliveries, then dead-lettered to `talos.dlq` (alert surfaced in Command Center). All consumers are idempotent keyed on `event_id` (processed IDs cached in Redis, 24 h TTL). PostgreSQL (`audit_events`, `agent_run_*`) is the durable record; RabbitMQ is transport only. Redis holds short-lived locks: `talos:lock:run:{project_id}:{branch}`, TTL = run timeout, released on terminal state.

12. Security and governance

Security is a core feature. Coding agents execute commands, modify files, and may interact with credentials. Design as if agent instructions and repository contents can be malicious.

12.1 Policy enforcement points (read this before implementing the policy engine)

Talos cannot intercept individual shell commands executed inside a third-party agent process. A naive `checkCommand(cmd)` service would never be in the execution path. Enforcement therefore happens at four real points:

Point	When	Mechanism
Pre-run	Before workspace prep	Validate task/config; refuse runs whose config or prompt targets blocked paths; verify approval requirements for `risk_level=HIGH` tasks.
Container/process level (**the hard boundary**)	During the run	Non-root execution, no Docker socket, CPU/memory/pids/time limits, `.env`/key files excluded from worktree by copy filter, network allow/deny per project (Phase 11), git wrapper blocking `push --force` and pushes to `default_branch`.
Agent-native hooks	During the run, where supported	ClaudeCodeAdapter configures Claude Code permission settings/hooks in the provider home to deny matching commands natively (`capabilities.supports_approval_hooks=true`).
Post-run scan (**the review gate**)	REVIEWING state	Scan the captured diff against `files.blocked_patterns` and `approval_required_patterns`, and the captured command log (from `TOOL_USE` events) against command patterns. Any match sets `review_status=RISK_FLAGGED`, marks the matching `git_changes.risk_flagged`, and the Review Center displays exactly what matched.

Command block/approval lists are *advisory* except where enforced natively or at the container boundary; their guaranteed effect is flagging the run for mandatory human scrutiny before anything is pushed or deployed.

12.2 Controls

Control	MVP requirement
Identity and auth	Single admin user seeded by migration from `TALOS_ADMIN_EMAIL`/`TALOS_ADMIN_PASSWORD`. Login issues a 24 h JWT signed with `TALOS_JWT_SECRET`. All `/api/v1/**` except `/auth/login` and `/webhooks/**` require it. OAuth/SSO later.
RBAC	Roles OWNER/MAINTAINER/REVIEWER/VIEWER exist in the schema; MVP runs owner-mode (every check passes for the admin). Enforcement arrives with multi-user (post-MVP).
Policy engine	`policy.yaml` (global) merged with `talos.yaml rules` (per project). Patterns: file patterns are gitignore-style globs matched against changed paths; command patterns are matched as substrings of captured command lines. Applied at the four points in 12.1.

Control	MVP requirement
Secrets	`secret_values` table, AES-256-GCM with `TALOS_SECRETS_KEY`, accessed only by `dev.talos.secrets`. `secret_ref` UUIDs everywhere else. Never returned by any REST endpoint. Vault is a post-MVP swap.
Secret injection	A run receives only env vars explicitly listed for its project and approved; delivered process-env only, never written to disk.
Log masking	Every log path (runner, orchestrator, API) replaces occurrences of any injected secret value with `***` before persistence. Covered by tests.
Runner isolation	Non-root, no host filesystem access outside `/var/talos`, no Docker socket, per-run resource limits.
Audit trail	Task creation, prompt, agent selection, transitions, files changed, approvals, pushes, deploys — all written to `audit_events`.
Human approval	Required for: any push/PR (always), production deploys, database migrations, auth/security-path changes, secrets access changes, `risk_level=HIGH` tasks.

12.3 Example `policy.yaml`

```
commands:
  blocked:
    - "rm -rf /"
    - "chmod 777"
    - "docker system prune"
    - "curl | sh"
  approval_required:
    - "sudo"
    - "systemctl"
    - "ssh "
    - "psql"
    - "docker compose down"
    - "npm audit fix --force"
files:
  blocked_patterns:
    - ".env*"
    - "*.pem"
    - "id_rsa*"
    - "**/secrets/**"
  approval_required_patterns:
    - ".github/workflows/**"
    - "docker-compose*.yml"
    - "src/main/resources/application-prod*.yml"
    - "**/migrations/**"
    - "**/db/migration/**"
deploy:
  production_requires_approval: true
  staging_auto_deploy_allowed: false
```

13. Subscription vs API-key execution strategy

Support both personal subscription-based CLI use and API-key-based automation, treated differently:

Mode	Use case	Recommendation
Personal subscription CLI	You run Talos for yourself on your workstation or VPS, using your own authenticated Claude Code/Codex CLI session.	Acceptable for personal workflow. Credentials live in the provider home (Section 7.5); never expose this mode to other users. `provider_auth_mode=subscription_local`.
API key / BYOK	Server-side automation, CI, monetized product, team mode.	Preferred for productized workflows. Each provider key explicitly configured as an integration credential and billed per provider terms. `provider_auth_mode=api_key`.
Hybrid local executor	A user runs a local runner on their own machine connecting back to the dashboard.	Best future commercial model; out of MVP scope.
Hosted multi-tenant subscription proxy	Your server uses customers' subscription logins for automated work.	Avoid unless explicitly supported and approved by the provider. Not implemented.

The UI shows which auth mode each run used (`agent_runs.provider_auth_mode`), and integrations configuration labels subscription-mode providers as "personal use only".

14. Project configuration spec

Every managed repo should include a `talos.yaml`; the platform can also hold the equivalent config in the database (pasted via `sync-config`) for repos where committing config is undesirable. Schema (enforced by `packages/project-config-schema/talos.schema.json`):

Field	Type	Required	Notes
`project.name`	string	yes	Display name
`project.type`	enum: `spring-boot,angular,python,node,static`	yes	Selects worker image later
`project.repo`	string (git URL)	yes	
`project.default_branch`	string	no (default `main`)	
`stack.language` / `stack.runtime` / `stack.framework` / `stack.package_manager`	strings	no	Informational + prompt context
`commands.install` / `commands.test` / `commands.build` / `commands.lint`	string or null	`test` required	Executed via CustomShellAdapter in the workspace
`agents.preferred`	adapter key	no (default `claude-code`)	
`agents.allowed`	list of adapter keys	no (default: all registered)	`start-run` rejects keys outside this list
`rules.require_approval_for`	list	no	Merged with global policy.yaml
`rules.forbidden`	list	no	Merged with global policy.yaml
`context.docs`	list of repo paths	no	Injected into prompts (Section 7.3)
`context.important_paths` / `context.ignore_paths`	lists	no	`ignore_paths` feeds the worktree copy filter
`deploy.provider` / `deploy.app_id` / `deploy.environment` / `deploy.approval_required`	strings/bool	no	Phase 10

Example:

```yaml
project:
  name: example-backend
  type: spring-boot
  repo: git@github.com:org/example-backend.git
  default_branch: main
stack:
  language: java
  runtime: java-21
  framework: spring-boot
  package_manager: maven
commands:
  install: "./mvnw dependency:resolve"
  test: "./mvnw test"
  build: "./mvnw clean package -DskipTests=false"
  lint: null
agents:
  preferred: claude-code
  allowed: [claude-code, opencode, custom-shell]
rules:
  require_approval_for: [production_deploy, database_migration, auth_changes,
                         security_config, dependency_major_upgrade]
  forbidden: [commit_secrets, modify_env_files, force_push_main]
context:
  docs: [docs/architecture.md, docs/api-guidelines.md]
  important_paths: [src/main/java, src/main/resources]
  ignore_paths: [target, node_modules, .env]
deploy:
  provider: dokploy
  app_id: backend-app-id
  environment: production
  approval_required: true
```

Invalid config is rejected at `sync-config` time with field-level errors; a project without valid active config cannot start runs.

15. Dashboard and UX scope

Angular 22, standalone components, signals, Angular Material. Routes:

Route	Screen	Key components	First version behavior
`/`	Command Center	`CommandCenterPage`, `RunCard`, `ApprovalCard`	Cards for active runs, approvals waiting, failed builds, recently completed tasks.
`/projects`	Projects	`ProjectListPage`, `ProjectFormDialog`	List/create/edit; stack, repo health, latest run.
`/projects/:id`	Project Detail	`ProjectDetailPage`, `ConfigPanel`, `RunHistoryTable`	Overview, tasks, runs, config version history, repository links.
`/board`	Kanban	`BoardPage`, `TaskColumn`, `TaskCard`, `TaskDrawer`	CDK drag-drop between the six columns; start agent run from a card; agent/status badges. Optimistic move with rollback on 422.

Route	Screen	Key components	First version behavior
`/tasks/new` (dialog)	New Task	`TaskFormDialog`	Description, project picker, priority, agent choice (from `agents.allowed`), risk level.
`/runs/:id`	Run Detail	`RunDetailPage`, `StatusTimeline`, `LiveLogPane`, `StepList`	Timeline, live SSE logs with auto-scroll and virtual scrolling (logs can reach tens of thousands of lines), current step, changed files, artifacts, test results, cancel button.
`/review/:runId`	Review	`ReviewPage`, `FileTree`, `UnifiedDiffView`, `RiskPanel`, `ApprovalActions`	Unified diff text (rich side-by-side viewer later), risk flags with matched patterns, test results, approve/reject/request-changes with notes.
`/integrations`	Integrations	`IntegrationsPage`, `IntegrationFormDialog`	Git provider, Dokploy, model providers, auth-mode labels.
`/login`	Login	`LoginPage`	Email/password → JWT in memory + refresh on load.

UX rules: every mutating action shows a snackbar with success/failure; every list has empty/loading/error states; approval actions require a confirmation dialog restating what will happen (e.g. "Approving will push branch X and open a PR against main").

16. Implementation phases

Global rules: every phase ends with all tests green, `docker compose -f infra/docker-compose.dev.yml up` working, and `docs/phase-reports/phase-N-report.md` (what works, what is stubbed, deviations). No phase starts until the previous phase's acceptance criteria pass. Convert each phase's tasks into tickets of at most half a day each before starting it.

Phase 0 — Repository and tooling setup

- **Goal:** Monorepo skeleton, dev infra, contract scaffolding, CI.
- **Files:** the tree from Section 5 (minus deferred dirs); `infra/docker-compose.dev.yml` (postgres:17, rabbitmq:4.1-management, redis:7 with healthchecks); `.env.example` per app (Appendix A); `packages/contracts/openapi.yaml` stubs; root `README.md`; CI workflow building all apps.
- **Tasks:** scaffold Angular 22 app, Spring Boot 4.1 app (package `dev.talos`, Maven), two `uv` Python projects; pin versions in `docs/architecture.md`.
- **Acceptance:** compose starts infra healthy; `./mvnw verify`, `ng build`, `uv run pytest` all pass (empty suites allowed); `grep -ri agentos .` returns nothing; CI green.

Phase 1 — Backend foundation (auth, errors, audit)

- **Goal:** Running API with login, JWT security, error envelope, audit writer.
- **Files:** `dev.talos.auth`, `dev.talos.audit`, `dev.talos.common`; `V001__users_and_audit.sql`.
- **Tasks:** seed admin from env; `POST /auth/login`; JWT filter on all routes; `@ControllerAdvice` error envelope; `/actuator/health` public.
- **Acceptance:** login with seeded credentials returns a JWT; unauthenticated request → 401 with the standard error body; every login writes an `audit_events` row.
- **Tests:** MockMvc login success/failure and 401 shape; audit row asserted via Testcontainers PostgreSQL.

Phase 2 — Database and migrations

- **Goal:** Full MVP schema from Section 9.2 (and nothing from 9.4).
- **Files:** `V002__core_schema.sql`; JPA entities + repositories per package.
- **Acceptance:** clean `flyway migrate` on an empty DB; `ddl-auto=validate` passes; CHECK constraints match Section 9.3 exactly.
- **Tests:** repository round-trips per entity; migrate-twice idempotency.

Phase 3 — Project registry

- **Goal:** Projects CRUD + talos.yaml parsing, API and UI.
- **Files:** `dev.talos.projects`; `talos.schema.json`; Angular `/projects`, `/projects/:id`, generated API client.
- **Tasks:** CRUD endpoints; `sync-config` validating against the schema, storing versioned `project_configs`, enforcing single active version.

- **Acceptance:** create/list/edit a project in the UI; invalid talos.yaml rejected with field-level errors; config version history retained and visible.
- **Tests:** parser units (valid, missing required, unknown agent key); MockMvc CRUD; one Angular component test.

Phase 4 — Task board

- **Goal:** Tasks CRUD + Kanban.
- **Files:** `dev.talos.tasks`; Angular `/board`, task form/drawer.
- **Tasks:** server-side transition validation; `board_position` ordering; `move` endpoint; audit on all mutations.
- **Acceptance:** UI-created task appears in Backlog; drag to Ready persists across reload; Backlog→Done rejected with 422; audit rows written.
- **Tests:** transition-matrix units; move MockMvc; board drag component test.

Phase 5 — Agent run lifecycle (API side)

- **Goal:** Run records, state machine, event publishing, log ingest, SSE. No execution yet.
- **Files:** `dev.talos.runs`, `dev.talos.events`; `/internal/v1` filter; Redis→SSE bridge.
- **Tasks:** `start-run` creating CREATED→QUEUED and publishing schema-valid `task.run.requested`; internal status/steps/logs endpoints; per-state `timeout_at`; scheduled reaper.
- **Acceptance:** `start-run` yields a RabbitMQ message validating against its JSON Schema; logs POSTed internally appear on an open SSE connection; illegal transition → 422 + audit; reaper fails an expired run in test.
- **Tests:** exhaustive transition tests; publisher integration (Testcontainers RabbitMQ); SSE integration.

Phase 6 — Orchestrator and runner supervisor (dummy flow)

- **Goal:** Full pipeline with `CustomShellAdapter`: consume → workspace → execute → tests → diff → WAITING_APPROVAL.
- **Files:** `apps/orchestrator`, `apps/runner-supervisor`, `packages/agent-adapter-spec` (ABC + CustomShellAdapter + contract tests); `scripts/smoke.sh`.
- **Tasks:** pipeline with status updates at each step; Redis lock; log batching; workspace manager (clone, worktree, branch naming, copy filter); diff capture; cleanup endpoint; crash-recovery behavior from Section 6.3.
- **Acceptance:** against a local fixture repo, `start-run` walks CREATED→… →WAITING_APPROVAL; the worktree exists on the named branch; the dummy script's change appears in `git_changes`; logs visible in DB and via SSE; a second concurrent start on the same project/branch is rejected by the lock; cancel mid-run kills the process tree and lands CANCELLED.
- **Tests:** pipeline with mocked runner; workspace tests on a temp git repo; timeout and cancel tests; `smoke.sh` in CI.

Phase 7 — First real agent adapter: ClaudeCodeAdapter

- **Goal:** A real coding agent completes a real task end to end.
- **Files:** `ClaudeCodeAdapter`; prompt assembler in the orchestrator; provider-home bootstrap docs (`docs/provider-auth.md`).
- **Tasks:** verify current headless flags against Claude Code docs; stream-json event parsing into `TOOL_USE`/`LOG`; isolated `HOME`/`CLAUDE_CONFIG_DIR`; both auth modes; native permission-hook configuration as a policy enforcement point; kill-tree on stop.
- **Acceptance:** a task like "add a /hello endpoint" against a fixture Spring Boot repo produces an agent-authored diff, tests run, run reaches WAITING_APPROVAL; contract tests (7.2) pass for both CustomShellAdapter and ClaudeCodeAdapter; recorded-output parsing test keeps CI independent of a live agent.
- **Tests:** contract suite; prompt-assembly units; stream-json parser against recorded fixtures.

Phase 8 — Review and approval flow

- **Goal:** Review Center + approvals gating everything downstream.
- **Files:** `dev.talos.approvals`, `dev.talos.policy`; Angular `/runs/:id` and `/review/:runId`.
- **Tasks:** auto-create approval on WAITING_APPROVAL; post-run policy scan (12.1 point 4); approve/reject/request-changes actions; task status sync; audit.
- **Acceptance:** a completed run appears in Review Center with correct diff and test results; a run touching `.env` is RISK_FLAGGED with the matched pattern displayed; **no push/PR/deploy is possible without an APPROVED approval row — enforced server-side and proven by a test**.
- **Tests:** policy-scan units per pattern class; approval workflow MockMvc; UI approval action test.

Phase 9 — Git push and PR workflow

- **Goal:** Approval → commit → push → GitHub PR, recorded and linked.
- **Files:** GitHub client in `dev.talos.integrations`; push/PR pipeline step; credential injection via secret refs.
- **Tasks:** commit convention (8.4); push-after-approval only; PR body template; NEEDS_REBASE flag on rejected push; PR URL in UI.
- **Acceptance:** approving a run against a scratch GitHub repo yields a real PR with correct body; `pull_requests` row stored; credentials absent from every log (masking test); unapproved run cannot push (test).
- **Tests:** GitHub client against a mock server; masking tests; optional manual e2e against a scratch repo.

Phase 10 — Dokploy integration

- **Goal:** Approval-gated deploy trigger + Talos's own production runbook.
- **Files:** `DokployClient` behind a `DeployProvider` interface (no-op default); `project_environments` migration; `infra/dokploy/`; production Dockerfiles per service; `docs/deployment.md`.

- **Tasks:** trigger deploy for `deploy.app_id` only with satisfied approval; deploy status polling; `deploy.*` events; rollback documentation (redeploy previous image tag).
- **Acceptance:** deploy button disabled without approval; mock-Dokploy integration test covers success and failure; a fresh operator can deploy Talos to Dokploy from `docs/deployment.md` alone.
- **Tests:** DeployProvider contract tests; approval-gate test.

Phase 11 — Hardening and security

- **Goal:** Close the isolation and abuse-case gaps.
- **Tasks:** per-run Docker execution using `workers/` images (non-root, no docker.sock, cgroup limits, network policy) behind the unchanged runner HTTP contract; secret-masking sweep; rate limits; retention job verification; dependency + container scanning in CI; `docs/security-model.md` threat model; PostgreSQL backup/restore procedure, executed once as a drill.
- **Acceptance:** security suite passes — agent cannot read a planted `.env` outside the workspace; blocked-pattern diff flagged; runner container cannot reach the Docker daemon; run hard-killed at timeout; restore drill documented.

Post-MVP (Phases 12+)

The MVP ends at Phase 11. The phases below are sequenced and specified to the same headings as Phases 0–11, but at lower resolution: each receives a full ticket-level breakdown in a plan revision immediately before it starts. Phase numbers 12 and 13 are fixed by cross-references elsewhere in this document; 14–16 are provisional and may be reordered. Every hard constraint from Section 20 continues to apply — in particular, nothing outside `packages/agent-adapter-spec` may couple to a provider, `talos-api` remains the only PostgreSQL writer, and no new communication path is added to the Section 4.3 spine.

Phase 12 — Additional adapters and remote triggers

Two independent tracks, gated together. Track A proves the agent-agnostic claim with real second and third adapters; Track B adds chat-based task intake and notifications.

Track A — adapters (`OpenCodeAdapter`, `CodexCliAdapter`, `OpenHandsAdapter`).

- **Goal:** Replace the Phase 6 `NotImplementedError` stubs with working adapters, per the fixed order in Section 7.4. Per that section, each adapter's first ticket is verifying current invocation flags/APIs against provider documentation and recording the verified CLI version in the module docstring — none of the invocations below may be assumed still-accurate at implementation time.
- **Files:** adapter modules in `packages/agent-adapter-spec`; recorded stream fixtures per adapter; provider homes `/var/talos/provider-homes/{opencode,codex,openhands}/` (Section 7.5).
- **Tasks:**
  - `OpenCodeAdapter` — wrap the OpenCode CLI in non-interactive mode with JSON output; map its event stream to `TOOL_USE`/`LOG` adapter events; model/provider selection lives in config inside the provider home, so Talos stays provider-flexible without new schema. `api_key` auth mode only.

    - `CodexCliAdapter` — wrap `codex exec` non-interactive mode, parsing its JSON/JSONL output into adapter events. `provider_auth_mode=api_key` is the automation path; ChatGPT sign-in is `subscription_local`, personal use only, never exposed to other users (Section 13).
    - `OpenHandsAdapter` — the first non-subprocess adapter: an HTTP/ACP client against a locally deployed OpenHands instance. It still implements the same `AgentAdapter` ABC; `execute()` proxies the remote event stream into the standard event iterator, and `stop()` cancels the remote session. Because OpenHands sandboxes execution itself, this adapter may later serve as an alternative execution backend, but in Phase 12 it runs in the standard workspace model. This ticket is last in the phase and may slip to a later phase without blocking the gate (Section 7.4 lists it as "Phase 12+").
    - Capability check before each launch (CLI present in image, version compatible, credentials present in provider home), failing the run with a clear `SYSTEM` log line rather than a mid-run crash.
- **Acceptance:** the Section 7.2 contract suite passes for every registered adapter in CI (recorded fixtures for the real agents); the Phase 6 fixture-repo smoke run reaches `WAITING_APPROVAL` with each new adapter with *only* `assigned_agent_key` changed and zero orchestrator/runner code modified; each adapter module docstring names its verified CLI/API version.
- **Tests:** contract suite per adapter; fixture-drift parser tests; capability-check unit tests.

Track B — remote triggers (Telegram first, then WhatsApp).

- **Goal:** Create and monitor tasks from chat without weakening any governance rule. Approval *decisions* stay in the dashboard — chat receives notifications with deep links, never approve/reject commands (a spoofed or stolen chat session must not be able to authorize a push or deploy).
- **Files:** `apps/telegram-adapter`, then `apps/whatsapp-adapter` — each its own Docker image and Dokploy app, per the Section 5 tree; a shared inbound-command schema in `packages/contracts` so the API surface does not grow per channel.
- **Design:** each trigger service is an ordinary API client plus a notifier — exactly the two roles the existing spine already provides for. Inbound: it authenticates to the public REST API with a dedicated service-account JWT (least-privilege: task create/read only; it never touches `/internal/v1`). Outbound: it consumes the "API → notifiers" routing keys from Section 11 (`approval.requested`, `pr.created`, `run.status.changed`) on its own quorum queue, idempotent on `event_id` like every consumer.
- **Tasks:** Telegram Bot API integration (webhook or long-poll); strict allow-list of operator chat IDs, all other senders ignored and audited; bot token stored in `secret_values` as an integration credential and masked in all logs; commands for "create task in project X" (persisting `tasks.source='TELEGRAM'`), "status of task/run", "list pending approvals"; notification messages carrying dashboard deep links. WhatsApp follows as a second implementation of the same command schema via the WhatsApp Business Cloud API, with signed-webhook verification.
- **Acceptance:** a message from an allow-listed chat creates a `BACKLOG` task with `source=TELEGRAM` visible on the Kanban board; a message from any other chat ID produces no task and an audit row; `approval.requested` on a run produces a chat notification within seconds; the bot token never appears in any log; no approval can be granted from chat.

- **Tests:** command parser units; allow-list enforcement; end-to-end against a mocked Bot API; notifier consumption integration test.

Phase 13 — Memory (`memory_documents`/`memory_chunks` + pgvector)

- **Goal:** Retrieval-augmented project memory so runs stop rediscovering project context. This phase creates the tables deferred in Section 9.4 — they must not exist before it.
- **Files:** migration enabling the pgvector extension and creating `memory_documents` and `memory_chunks` (chunk text, embedding vector, source reference, project FK); `dev.talos.memory`; an orchestrator prompt-assembly extension.
- **Design:** the API owns ingestion, embedding, and retrieval (constraint: sole DB writer). Embeddings are computed by the API's memory service through a configured embedding provider key (BYOK; never a subscription credential). The orchestrator retrieves via a new internal endpoint (`GET /internal/v1/projects/{id}/memory/search`) and injects results into the Section 7.3 prompt as a new stage between project context (2) and the task (3); the assembled prompt including memory is still persisted to `agent_runs.prompt`, so auditability is unchanged.
- **Tasks:** ingestion pipeline for completed-run summaries/diff digests, operator notes, and `context.docs`; chunking + embedding with a per-project token budget for injected memory; retrieval strictly project-scoped — no cross-project reads; memory text passes the same secret-masking sweep as logs; a per-project switch (`talos.yaml`) that disables memory entirely.
- **Acceptance:** completed runs and ingested docs produce embedded chunks; retrieval returns only same-project chunks; a new run's prompt contains relevant memory within budget; with memory disabled, the assembled prompt is byte-identical to pre-Phase-13 output.
- **Tests:** chunker/budget units; project-isolation test; prompt-assembly regression test against recorded Phase 7 prompts.

Phase 14 — Cost tracking and recommendations

- **Goal:** Per-provider cost visibility, then advisory recommendations built on that history. Cost tracking lands first because recommendations consume its data.
- **Cost tracking:** extend `AdapterResult` with normalized usage metadata (input/output tokens, provider-reported cost, model); persist per run (new `agent_runs` columns or a `run_costs` table — decided at phase start); aggregate per provider / project / month; a dashboard cost widget. `subscription_local` runs record token counts with a null dollar cost and are labeled as such (Section 13) — never estimate a price for subscription usage.
- **Recommendations:** advisory-only signals computed from run history (outcomes, durations, review flags, costs): suggested agent for a task, risk flag ("this file area has failed review twice"), cheapest-capable-agent hint. Surfaced in the task form and Command Center. Recommendations never auto-select an agent and never auto-execute anything — the operator always confirms.
- **Acceptance:** after fixture runs on two providers, monthly per-provider totals match the fixtures' known usage exactly; a run with no usage metadata degrades gracefully (row with nulls, no pipeline failure); ignoring a recommendation changes no behavior.

- **Tests:** usage-normalization units per adapter; aggregation queries; recommendation-signal units on seeded history.

Phase 15 — Multi-user RBAC enforcement

- **Goal:** Replace MVP owner-mode (Section 12.2: every check passes for the admin) with real enforcement of the roles that have existed in the schema since Phase 1: `OWNER`, `MAINTAINER`, `REVIEWER`, `VIEWER` (Section 9.3).
- **Tasks:** user management endpoints (create/invite, deactivate, role assignment — OWNER only); role claim in the JWT; a server-side authorization matrix — `VIEWER` read-only; `REVIEWER` adds approve/reject/request-changes; `MAINTAINER` adds project/task CRUD and run start; `OWNER` adds integrations, secrets, user management, and deploy triggers; self-approval prohibition — the user who requested a run cannot approve it (OWNER may override; the override is audited); `audit_events.actor` verified on every mutation; UI hides actions the current role cannot perform, but enforcement is server-side.
- **Acceptance:** role × endpoint-class matrix covered by tests; a `VIEWER` attempting an approval receives 403 plus an audit row; a fresh single-user install behaves exactly as before (seeded admin is `OWNER`); no endpoint relies on UI hiding for protection.
- **Tests:** exhaustive matrix (MockMvc per role); self-approval test; migration test that existing installs map the admin to `OWNER`.

Phase 16 — MinIO artifact storage

- **Goal:** Move run artifacts (transcripts, patches, test reports, generated docs — Section 4.2) from the local volume to S3-compatible object storage, behind an interface so the local volume remains the default for small installs.
- **Files:** an `ArtifactStore` interface in `dev.talos` with `LocalVolumeArtifactStore` (current behavior, default) and `MinioArtifactStore` implementations; `talos-minio` Dokploy app (the Section 18 table already reserves its row).
- **Tasks:** bucket layout per artifact class; uploads by the API only (runner/orchestrator continue to hand artifacts to the API — no new communication path, and worker containers never receive object-store credentials); downloads streamed through the existing REST endpoints (no browser-visible presigned credentials in the first cut); the Phase 11 retention sweep extended to delete from the object store; a one-shot migration command copying existing local artifacts.
- **Acceptance:** with MinIO configured, a run's artifacts land in the bucket and download byte-identical through the same REST endpoints; with it unconfigured, behavior is exactly pre-Phase-16; retention removes expired artifacts from both stores; MinIO credentials appear in no log and no worker environment.
- **Tests:** `ArtifactStore` contract test run against both implementations; retention integration test; migration round-trip test.

17. Testing strategy

Layer	Tests	Gate
Backend unit	State transitions (exhaustive matrix), policy scanner, approval rules, config parser, secret encryption	Every PR
Backend integration	Flyway migrations, RabbitMQ publish/consume (Testcontainers), GitHub/Dokploy mocks, SSE	Every PR
Frontend	Kanban movement, run detail rendering, approval actions, form validation	Every PR
Orchestrator	Pipeline state machine, adapter mocks, retry/timeout/cancel, crash recovery	Every PR
Runner	Workspace creation, branch naming, copy filter, diff capture, log streaming, cleanup, kill-tree	Every PR
Adapter contract	Suite 7.2 against every registered adapter (recorded fixtures for real agents)	Every PR
Security	Blocked commands flagged, secret masking, file-pattern blocking, no write to default branch, container isolation (Phase 11)	Every PR from Phase 8
End-to-end smoke	`scripts/smoke.sh`: create project → task → run → CustomShellAdapter → diff → approval requested	Every PR from Phase 6

CI runs all layers on every pull request; a phase gate additionally requires its acceptance criteria demonstrated and the phase report written.

18. Dokploy deployment plan

Deploy each service separately in Dokploy, backed by persistent volumes for PostgreSQL, RabbitMQ, Redis, and `/var/talos` (workspaces + provider homes + artifacts). Start with one VPS and low concurrency.

Dokploy app	Image source	Health check	Suggested initial resources
`talos-web`	`apps/web/Dockerfile` (nginx serving the Angular build)	`GET /`	0.25–0.5 vCPU, 256–512 MB
`talos-api`	`apps/api/Dockerfile`	`GET /actuator/health`	1 vCPU, 1–2 GB
`talos-orchestrator`	`apps/orchestrator/Dockerfile`	process liveness	0.5–1 vCPU, 512 MB–1 GB
`talos-runner-supervisor`	`apps/runner-supervisor/Dockerfile`	`GET /health`	1–2 vCPU, 1–2 GB + worker limits
`talos-postgres`	`pgvector/pgvector:pg17`	`pg_isready`	1–2 vCPU, 2–4 GB

Dokploy app	Image source	Health check	Suggested initial resources
`talos-rabbitmq`	`rabbitmq:4.1-management`	`rabbitmq-diagnostics ping`	0.5–1 vCPU, 512 MB–1 GB
`talos-redis`	`redis:7`	`redis-cli ping`	0.25–0.5 vCPU, 256–512 MB
`talos-minio`	optional, later	—	—

Initial concurrency: `TALOS_MAX_ACTIVE_RUNS=1`, `TALOS_MAX_WORKSPACE_AGE_DAYS=7`, `TALOS_RUN_TIMEOUT_MINUTES=60`.

Operational rules: deploy order postgres → rabbitmq/redis → api (runs migrations on boot) → orchestrator/runner → web. Environment variables map 1:1 from Appendix A into Dokploy app settings. **Rollback:** redeploy the previous image tag (images are tagged with the git SHA); Flyway migrations must be backward-compatible one release back so an API rollback never requires a schema rollback. A failed deploy (health check never green) keeps the previous container running; investigate via Dokploy logs before retrying. TLS terminates at Dokploy's Traefik with a dedicated domain per service (`talos.example.com` for web, `api.talos.example.com`).

19. Risks and mitigations

Risk	Mitigation
Provider subscription terms change or block automation	API-key/BYOK path is first-class; subscription mode is personal-use-only and isolated per Section 13.
Agent CLIs change flags/output between versions	Adapters record their verified CLI version; contract tests + recorded-fixture parsers detect drift; capability checks before launch.
VPS resource exhaustion	Strict concurrency (1 run), cgroup limits, timeouts, retention cleanup, build/worker separation.
Security incident through malicious repo/config	Sandboxing, four-point policy enforcement (12.1), no automatic secret mounts, audit log, per-project trust levels later.
Scope creep	MVP = three stack profiles, one real adapter (Claude Code), GitHub only, no Telegram/WhatsApp/Odoo/Flutter until Phase 12+.
Difficult diff/review UX	Unified diff text first; rich side-by-side viewer later.
Monetization conflicts with provider subscriptions	Commercial version uses API keys, BYOK, or user-owned local runners — never shared subscription auth.
Orchestrator/API contract drift	OpenAPI + event JSON Schemas in `packages/contracts` validated by both sides' tests in CI.

20. Claude Code execution prompt

Use the following prompt to have Claude Code implement the platform incrementally. It is also maintained as `docs/talos_claude_code_execution_prompt.md`.

You are Claude Code acting as the implementation engineer for **Talos**, a self-hosted, web-based, agent-agnostic control plane that orchestrates existing coding agents. Your single source of truth is this implementation plan (Revision 2). Do not invent architecture: every table, endpoint, event, enum, and state transition you create must match Sections 7–11 verbatim. If a contract is ambiguous or contradicts observed reality (for example, a CLI flag changed), stop and ask rather than guessing.

Hard constraints:

1. Do not build a new LLM or coding agent. Talos only orchestrates existing agents through the `AgentAdapter` interface in `packages/agent-adapter-spec`.
2. Do not couple anything outside the adapter package to a specific agent provider. The orchestrator must run identically with `CustomShellAdapter`.
3. Implement adapters in this order and no other: `CustomShellAdapter` (Phase 6) → `ClaudeCodeAdapter` (Phase 7). `OpenCodeAdapter`, `CodexCliAdapter`, `OpenHandsAdapter` remain `NotImplementedError` stubs until Phase 12.
4. Monorepo `talos/`, but each app in `apps/` builds its own Docker image and is deployable alone. No shared runtime state except PostgreSQL, RabbitMQ, and Redis.
5. The Spring Boot API is the only writer to PostgreSQL. The orchestrator and runner supervisor mutate state exclusively through `/internal/v1` with the service token. Neither Python service opens a database connection.
6. Safe defaults everywhere: runs execute only on `agent/task-<id>-<slug>` branches in isolated worktrees; nothing is pushed, PR'd, or deployed without an APPROVED approval row, enforced server-side and covered by a test.
7. Never expose production secrets to agent workers. Runners receive only enumerated injected env vars; provider credentials live in isolated provider homes outside every workspace; all injected values are masked in every log path.
8. Do not implement any deploy trigger (Phase 10) before the review/approval flow (Phase 8) is complete and tested. Until then `DeployProvider` is an interface with a no-op implementation.
9. Naming is canonical: product Talos, package `dev.talos`, services `talos-api|web|orchestrator|runner-supervisor`, exchange `talos.events`, env prefix `TALOS_`, config file `talos.yaml`. Before every commit run `grep -ri agentos .` — it must return nothing.
10. The system stays self-hostable: `docker compose -f infra/docker-compose.dev.yml up` must work at the end of every phase; production runs on Dokploy per Section 18.

Process: Work strictly phase by phase (Section 16, Phase 0 → 11). Within a phase, complete one ticket at a time (tickets ≤ half a day). A phase is done only when its acceptance criteria pass, all tests are green, and you have written `docs/phase-reports/phase-N-report.md` stating what works, what is stubbed, and any deviations — deviations require sign-off before you continue. Write tests alongside code, not after the phase: unit tests for state transitions, the policy scanner, and the config parser; contract tests for every adapter; the end-to-end smoke script must pass in CI from Phase 6 onward.

Implementation log: Maintain `docs/initial_implementation_log.md` for the entire initial build, appending one entry per completed ticket, newest first; never rewrite or delete earlier entries — if a later change reverses one, say so in the new entry. Each entry has a `##`

`<date> — <short title>` heading, an **Ask** stating what was requested, and always a **Verification** section stating exactly which checks were run with their results and which were not, with reasons. Beyond that, include only the sections necessary and relevant at the time — e.g. **Root cause**, **Changed (backend)** / **Changed (frontend)** with per-file bullets naming the file and the *why* behind non-obvious choices, **Coverage**, **Notes** for semantics and design rationale, **Known blockers / follow-ups**. The full worked example of this format is in `docs/talos_claude_code_execution_prompt.md`.

Start now with Phase 0.

21. References

- OpenHands homepage and positioning: https://www.openhands.dev/
- OpenHands Docker sandbox guide: https://docs.openhands.dev/sdk/guides/agent-server/docker-sandbox
- OpenHands Agent Canvas repository: https://github.com/OpenHands/agent-canvas
- OpenHands ACP agents documentation: https://docs.openhands.dev/openhands/usage/agent-canvas/acp-agents
- Vibe Kanban GitHub repository: https://github.com/BloopAI/vibe-kanban
- Vibe Kanban documentation: https://vibe-kb.com/docs/getting-started/
- Nimbalyst GitHub repository: https://github.com/nimbalyst/nimbalyst
- OpenClaw ACP agents documentation: https://docs.openclaw.ai/tools/acp-agents
- OpenAI Codex authentication documentation: https://developers.openai.com/codex/auth
- Claude Code Agent SDK overview and third-party auth note: https://code.claude.com/docs/en/agent-sdk/overview
- Claude Code authentication documentation: https://code.claude.com/docs/en/authentication
- Claude Code headless/CLI reference (verify adapter flags here): https://code.claude.com/docs/en/cli-reference
- Claude Code hooks (native policy enforcement point): https://code.claude.com/docs/en/hooks
- BuilderMethods Agent OS: https://buildermethods.com/agent-os

Appendix A: Environment variable reference

Every variable ships with a commented entry in the relevant `.env.example`. No service reads variables outside its own list.

talos-api

Variable	Example	Purpose
`TALOS_DB_URL`	`jdbc:postgresql://postgres:5432/talos`	PostgreSQL JDBC URL
`TALOS_DB_USER` / `TALOS_DB_PASSWORD`	`talos` / —	Database credentials
`TALOS_RABBITMQ_URL`	`amqp://talos:...@rabbitmq:5672`	Queue connection
`TALOS_REDIS_URL`	`redis://redis:6379`	Locks + log pub/sub
`TALOS_JWT_SECRET`	64+ random chars	JWT signing key
`TALOS_INTERNAL_API_TOKEN`	64+ random chars	Shared service token for `/internal/v1`
`TALOS_ADMIN_EMAIL` / `TALOS_ADMIN_PASSWORD`	—	Seeded admin (password rotated after first login)
`TALOS_SECRETS_KEY`	32-byte base64	AES-256-GCM key for `secret_values`
`TALOS_GITHUB_WEBHOOK_SECRET`	—	HMAC verification for `/webhooks/github`
`TALOS_POLICY_FILE`	`/etc/talos/policy.yaml`	Phase 8: overrides the bundled default `policy.yaml` (Section 12.3); unset uses the classpath default
`TALOS_LOGIN_RATE_LIMIT_MAX_ATTEMPTS`	`10`	Phase 11 (Section 12.2): attempts allowed per client IP per window before `/api/v1/auth/login` returns 429
`TALOS_LOGIN_RATE_LIMIT_WINDOW_SECONDS`	`60`	Phase 11: fixed-window size (seconds) for the login rate limiter's Redis counter

talos-orchestrator

Variable	Example	Purpose
`TALOS_API_BASE_URL`	`http://talos-api:8080`	Internal API root
`TALOS_INTERNAL_API_TOKEN`	(same as API)	Service auth
`TALOS_RABBITMQ_URL` / `TALOS_REDIS_URL`	as above	Transport + locks
`TALOS_RUNNER_BASE_URL`	`http://talos-runner-supervisor:8081`	Runner HTTP root
`TALOS_MAX_ACTIVE_RUNS`	`1`	Concurrency cap
`TALOS_RUN_TIMEOUT_MINUTES`	`60`	Overall run timeout
`TALOS_WORKER_IMAGE_BASE`	`workers/base-agent-runner:latest`	Phase 11 (Section 8): per-run container image for projects whose `stackType` matches no specific runner
`TALOS_WORKER_IMAGE_JAVA`	`workers/java-runner:latest`	Image for `stackType: spring-boot`
`TALOS_WORKER_IMAGE_NODE`	`workers/node-runner:latest`	Image for `stackType: angular` or `node`
`TALOS_WORKER_IMAGE_PYTHON`	`workers/python-runner:latest`	Image for `stackType: python`
`TALOS_RETENTION_MAX_AGE_DAYS`	`7`	Phase 11 (Section 8.3): terminal runs older than this with no OPEN PR are deleted by the periodic retention sweep
`TALOS_RETENTION_INTERVAL_SECONDS`	`21600`	How often the retention sweep runs (default 6h)

talos-runner-supervisor

Variable	Example	Purpose
`TALOS_WORKSPACES_ROOT`	`/var/talos/workspaces`	Workspace volume
`TALOS_PROVIDER_HOMES_ROOT`	`/var/talos/provider-homes`	Credential isolation volume
`TALOS_INTERNAL_API_TOKEN`	(same)	Auth for artifact/log posts
`TALOS_MAX_WORKSPACE_AGE_DAYS`	`7`	Retention
`TALOS_RUN_WORKSPACES_VOLUME`	`talos_workspaces`	Phase 11 (Section 8): named Docker volume mounted (per-run, via `--mount ...,volume-subpath=...`) into each per-run container

Variable	Example	Purpose
`TALOS_RUN_PROVIDER_HOMES_VOLUME`	`talos_provider_homes`	Named Docker volume for the per-run container's `/provider-home` mount
`TALOS_RUN_NETWORK`	`talos_run_network`	Isolated Docker network every per-run container joins instead of the compose default network -- no route to postgres/rabbitmq/redis/api
`TALOS_RUN_MEMORY_LIMIT`	`1g`	Per-run container `--memory`
`TALOS_RUN_CPU_LIMIT`	`1`	Per-run container `--cpus`
`TALOS_RUN_PIDS_LIMIT`	`256`	Per-run container `--pids-limit`
`TALOS_DOCKER_GID`	host-specific, e.g. `999`	Not read by the app itself -- consumed by `infra/*.yml`'s `group_add` so the container's non-root `talos` user can use the mounted `/var/run/docker.sock`. Find it with `stat -c '%g' /var/run/docker.sock` on the host.

talos-web

Variable	Example	Purpose
`TALOS_API_URL`	`https://api.talos.example.com`	Injected at container start into the served config

End of implementation plan — Revision 2, 2026-07-09.